

JC4003 Natural Language Processing

Lecture 6: Text Classification and Naïve Bayes

Dr Xiao Li xiao.li@abdn.ac.uk

Dr Ruizhe Li ruizhe.li@abdn.ac.uk

Dr Siwei Liu siwei.liu@abdn.ac.uk

Outline

Text Classification

The Naïve Bayes Classifier

Sentiment and Binary Naïve Bayes

Evaluate Language Models (part 3)

Text Classification

Text Classification

Text classification is the process of categorising written text into organised groups based on its content and context.

Image Classification



cat



smiling
female
portrait
curly hair
happy

Text Classification

Today has been an
absolutely wonderful
day filled with joy
and laughter.

positive

NLP and ML have
greatly improved movie
recommendation
systems, making them
more accurate and
user-friendly.

NLP
ML
Movie

we focus on this

Text Classification

Common text classification tasks include:

- Assigning subject categories, topics, or genres
- Spam detection
- Authorship identification
- Age/gender identification
- Language Identification
- Sentiment analysis
- ...

Text Classification: Definition

Input:

- a document d
- a fixed set of classes $\mathcal{C} = \{c_1, c_2, \dots, c_j\}$

Output:

- a predicted class $c \in \mathcal{C}$

Classification Methods

Hand-coded Rules (we will not introduce this)

- Rules based on combinations of words or other features.
- Its accuracy can be high, if rules carefully refined by expert, but **building and maintaining these rules are expensive**

Supervised Machine Learning (will be introduced today)

- Naïve Bayes, Logistic regression, Support-vector machines, k-Nearest Neighbors, ...

Deep Learning (will be introduced in following lectures)

- Deep learning models e.g. LSTM, Transformer, Large Language Models, ...

The Naïve Bayes Classifier

The Naive Bayes Classifier

Naive Bayes Classifiers are a group of probabilistic algorithms **based on Bayes' Theorem**, which **assume that the features of a dataset are independent of each other**.

Naive Bayes Classifiers are easy to implement and **highly effective** for tasks such as spam detection, sentiment analysis, and document classification.

Naive Bayes Classifiers rely on very simple representation of document e.g. use **bag of words**.

Bag of Words

Bag of Words is a **text representation method** used in NLP. It **converts text into a collection of individual words**, ignoring grammar and word order but keeping multiplicity.

John likes to watch
movies. Mary likes
movies too. Mary
also likes to watch
football games.

Text



Bag of Words
(a collection)

likes	3
to	2
watch	2
movies	2
Mary	2
John	1
too	1
also	1
football	1
games	1

Word Counting

Bag of Words

Each unique word in the text corpus is represented as a feature in a **fixed-size vector**, with the value of each feature typically being the **word's frequency** in the document.

Bag of Words can be any other representation methods.

likes	3	→	likes	0.19	→ $\begin{bmatrix} 0.19 \\ 0.13 \\ 0.13 \\ 0.13 \\ 0.13 \\ 0.13 \\ 0.06 \\ 0.06 \\ 0.06 \\ 0.06 \\ 0.06 \end{bmatrix}$
to	2		to	0.13	
watch	2		watch	0.13	
movies	2		movies	0.13	
Mary	2		Mary	0.13	
John	1		John	0.06	
too	1		too	0.06	
also	1		also	0.06	
football	1		football	0.06	
games	1		games	0.06	

The Naïve Bayes Classifier

Naive Bayes is a **probabilistic classifier**, meaning that for a document d , out of all classes $c \in \mathcal{C}$ the classifier returns the class $\hat{c}$ which has the maximum posterior probability given the document. We use the hat notation $\hat{\cdot}$ to mean "our estimate of the correct class".

The Naive Bayes classifier can be represented by:

$$\hat{c} = \underset{c \in \mathcal{C}}{\operatorname{argmax}} P(c|d)$$

By applying Bayes' rules, it gives:

$$\hat{c} = \underset{c \in \mathcal{C}}{\operatorname{argmax}} \frac{P(d|c)P(c)}{P(d)}$$

$$P(x|y) = \frac{P(y|x)P(x)}{P(y)}$$

Bayes' rules

We can then, drop the denominator $P(d)$:

$$\hat{c} = \underset{c \in \mathcal{C}}{\operatorname{argmax}} \underbrace{P(d|c)}_{\text{likelihood}} \underbrace{P(c)}_{\text{prior}}$$

The Naïve Bayes Classifier (cont.)

Without loss of generalisation, we can represent a document d as a set of features $w_1, w_2, \dots, w_n$. Since we represent documents as bags of words, $w_1, w_2, \dots, w_n$ are considered as words. Thus:

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(d|c)P(c) \quad \Leftrightarrow \quad \hat{c} = \underset{c \in C}{\operatorname{argmax}} \underbrace{P(w_1, w_2, \dots, w_n|c)}_{\text{likelihood}} \underbrace{P(c)}_{\text{prior}}$$

- **Bag of Words assumption:** Assume position doesn't matter.
- **Conditional Independence:** Assume the feature probabilities $P(w_i|c_i)$ are independent given the class c , i.e. $P(w_1, w_2, \dots, w_n|c) = P(w_1|c) \cdot P(w_2|c) \cdot \dots \cdot P(w_n|c)$

The two assumption gives:

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(w_1, w_2, \dots, w_n|c)P(c) = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{w \in \text{text}} P(w|c)$$

The Naïve Bayes Classifier (cont.)

The cumulative product can lead to numerical difficulties, Naive Bayes also uses the logarithmic form representation.

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} \left[\log_2 P(c) + \sum_{w \in \text{text}} \log_2 P(w|c) \right]$$

Notes:

1. Taking log doesn't change the ranking of classes! The class with highest probability also has highest log probability!
2. It's a linear model:

Just a max of a sum of weights: a linear function of the inputs

So naive bayes is a linear classifier

Training the Naïve Bayes Classifier

As a supervised model, its training requires a labelled corpus. The texts in this corpus are paired with their respective categories. E.g.:

Text	label
Today has been an absolutely wonderful day filled with joy and laughter.	positive
The service at the restaurant was terrible, and the food was bland and overpriced.	negative
...	...

Examples of a labelled corpus with sentiment labels

Training the Naïve Bayes Classifier (cont.)

To learn the probability $P(w_i|c_j)$, we compute as the fraction of times the word w appears among all words in all documents of topic c .

We first concatenate all documents with category c_j into one big "category c_j " text. Then we use the frequency of w in this concatenated document to give a maximum likelihood estimate of the probability:

$$\hat{P}(w_i|c_j) = \frac{\text{count}(w_i, c_j)}{\sum_{w \in V_{oc}} \text{count}(w, c_j)}$$

i.e. fraction of times word w_i appears among all words in "category c_j " text.

Training the Naïve Bayes Classifier (cont.)

What if we have seen no training documents with the word "fantastic" and classified in the topic **positive**? In such a case the probability for this feature will be **zero**.

$$\hat{P}(\text{"fantastic"}|\text{positive}) = \frac{\text{count}(\text{"fantastic"}, \text{positive})}{\sum_{w \in \text{Voc}} \text{count}(w, \text{positive})} = 0$$

NB: zero probabilities in the likelihood term for any class **will cause the probability of the class to be zero**, no matter the other evidence! The simplest solution is the add-one (Laplace) smoothing introduced in the last slides.

$$\hat{P}(w_i|c_j) = \frac{\text{count}(w_i, c_j) + 1}{\sum_{w \in \text{Voc}} (\text{count}(w, c_j) + 1)} = \frac{\text{count}(w_i, c_j) + 1}{|\text{Voc}| + \sum_{w \in \text{Voc}} \text{count}(w, c_j)}$$

Training the Naïve Bayes Classifier (cont.)

In addition to find $P(w|c)$, we also need to find $P(c)$. The value of $P(c)$ is the proportion of class c in the corpus.

For example, if the corpus contains 10 texts, with 3 being **positive**, then $P(\text{positive}) = 0.3$.

$$P(c) = \frac{\text{count}(doc \sim c)}{\text{count}(doc)}$$

where doc denotes each text in the corpus; $doc \sim c$ denotes the text, which labelled as c .

Testing the Naïve Bayes Classifier

Given the formula of the Naïve Bayes Classifier, when $P(c)$ and $P(w|c)$ are found for each c and each w , the model is ready to use.

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} \left[\log_2 P(c) + \sum_{w \in \text{text}} \log_2 P(w|c) \right]$$

So that given a text e.g. *text* = "Today has been an absolutely wonderful day filled with joy and laughter", Naïve Bayes Classifier simply convert every word to probability and try every candidate class, then, return the most likely classification.

Unknown words

What about unknown words

- that appear in our test data
- but not in our training data or vocabulary?

We ignore them

- Remove them from the test document!
- Pretend they weren't there!
- Don't include any probability for them at all!

Why don't we build an unknown word model?

- It doesn't help: knowing which class has more unknown words is not generally helpful!

Stop words

Some systems ignore stop words, which are **very frequent words** like "the" and "a".

- Sort the vocabulary by word frequency in training set
- Call the top 10 or 50 words the stopword list.
- Remove all stop words from both training and test sets as if they were never there!

But removing stop words doesn't usually help, so in practice most Naïve Bayes algorithms use all words and don't use stopwords .

Sentiment and Binary Naïve Bayes

A Worked Sentiment Example

Consider the small corpus:

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

A Worked Sentiment Example

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

4. Scoring the test item:

$$P(-)P(d|-) = \frac{3}{5} \cdot \frac{2}{34} \cdot \frac{2}{34} \cdot \frac{1}{34} = 6.1e^{-5}$$

$$P(+)P(d|+) = \frac{2}{5} \cdot \frac{1}{29} \cdot \frac{1}{29} \cdot \frac{2}{29} = 3.2e^{-5}$$

1. Prior from training:

$$P(-) = 3/5$$

$$P(+) = 2/5$$

3. Likelihoods from training:

$$\hat{P}(w_i|c_j) = \frac{\text{count}(w_i, c_j) + 1}{|Voc| + \sum_{w \in Voc} \text{count}(w, c_j)}$$

$$\begin{aligned} \hat{P}(\text{"predictable"}|-) &= \frac{1+1}{20+14} & \hat{P}(\text{"no"}|-) &= \frac{1+1}{20+14} & \hat{P}(\text{"fun"}|-) &= \frac{0+1}{20+14} \\ \hat{P}(\text{"predictable"}|+) &= \frac{0+1}{20+9} & \hat{P}(\text{"no"}|+) &= \frac{0+1}{20+9} & \hat{P}(\text{"fun"}|+) &= \frac{1+1}{20+9} \end{aligned}$$

2. Drop "with"

Optimising for Sentiment Analysis

For tasks like sentiment, **word occurrence** seems to be more important than **word frequency**.

- The occurrence of the word fantastic tells us a lot
- The fact that it occurs 5 times may not tell us much more.

Binary multinomial Naïve Bayes, or **binary Naïve Bayes**

- **For each text in the corpus, remove duplicate words.** E.g. "it was pathetic the worst part was the boxing scenes" $\Rightarrow$ "it was pathetic the worst part boxing scenes"
- NB: this is different than Bernoulli naive bayes; see the textbook at the end of the chapter.

Evaluate Language Models (part 3)

Evaluating Classifiers

How well does our classifier work? Let's first address binary classifiers:

- Is this email spam? Yes(+) or No(-)
- Is this post about Delicious Pie Company? Yes(+) or No(-)

We'll need to know

1. What **did** our classifier say about each email or post?
2. What **should** our classifier have said, i.e. the correct answer, usually as defined by humans ("gold label")

The Confusion Matrix

The first step is [constructing the confusion matrix](#), a table for visualising how an algorithm performs compared to the human gold labels.

We use two dimensions (system output and gold labels), and each cell represents a set of possible outcomes.

		<i>gold standard labels</i>	
		gold positive	gold negative
<i>system output labels</i>	system positive	true positive	false positive
	system negative	false negative	true negative

Accuracy On the Confusion Matrix

For **accuracy**: it measures the percentage of all observations (e.g., all emails or tweets in spam or pie examples) that our system labelled correctly.

Although accuracy might seem like a natural metric, we generally don't use it for text classification tasks.

		<i>gold standard labels</i>	
		gold positive	gold negative
<i>system output labels</i>	system positive	true positive	false positive
	system negative	false negative	true negative

$$\text{accuracy} = \frac{\text{tp} + \text{tn}}{\text{tp} + \text{fp} + \text{tn} + \text{fn}}$$

Accuracy On the Confusion Matrix (cont.)

Accuracy **doesn't work** well when we're dealing with **uncommon or imbalanced classes**.

Suppose we look at 1,000 social media posts to find Delicious Pie-lovers (or haters)

- 1 of them talk about our pie
- 999 are posts about something unrelated

Imagine the following simple classifier always output "not about pie".

$$accuracy = \frac{tp + tn}{tp + fp + tn + fn} = \frac{0 + 999}{0 + 1 + 999 + 0} = 99.9\%$$

Accuracy doesn't work for unbalanced classes!

Precision and Recall

Precision: % of selected items that are correct

Recall: % of correct items that are selected

		<i>gold standard labels</i>		
		gold positive	gold negative	
<i>system output labels</i>	system positive	true positive	false positive	precision = $\frac{tp}{tp+fp}$
	system negative	false negative	true negative	
		recall = $\frac{tp}{tp+fn}$		accuracy = $\frac{tp+tn}{tp+fp+tn+fn}$

Precision and Recall

Suppose we look at 1,000 social media posts to find Delicious Pie-lovers (or haters)

- 1 of them talk about our pie
- 999 are posts about something unrelated

$$precision = \frac{tp}{tp + fp} = \frac{0}{0 + 1} = 0\%$$

$$recall = \frac{tp}{tp + fn} = \frac{0}{0 + 999} = 0\%$$

F1

F1 is a combination of precision and recall.

It is the **harmonic mean of precision and recall**, where precision is the ratio of true positive results to all predicted positives, and recall is the ratio of true positive results to all actual positives.

The F1 score ranges from 0 to 1, with **1 indicating perfect precision and recall**, thus providing a balanced measure of the model's accuracy in identifying positive instances.

$$F1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

F-measure

F1 is a special case of the F-measure: weighted harmonic mean of precision and recall.

$$\text{HarmonicMean}(a_1, a_2, \dots, a_n) = \frac{n}{\frac{1}{a_1} + \frac{1}{a_2} + \dots + \frac{1}{a_n}}$$

F1 is a special case of F-measure with $\alpha = \frac{1}{2}$

$$F = \frac{1}{\frac{\alpha 1}{P} + \frac{(1 - \alpha) 1}{R}}$$

The harmonic mean of two values is closer to the minimum of the two numbers than the arithmetic or geometric mean, thus **weighing the lower of the two numbers more heavily**.

In other words, if precision and recall are far apart, F1 will be nearer to the lower value. Therefore, to achieve a high F1 score, one **must perform well on both precision and recall**.

More Than 2 Classes?

Lots of text classification tasks have more than two classes.

- Sentiment analysis (positive, negative, neutral)
- Named entities (person, location, organisation)

We can define precision and recall for multiple classes like this 3-way email task:

		<i>gold labels</i>			
		urgent	normal	spam	
<i>system output</i>	urgent	8	10	1	$\text{precision}_u = \frac{8}{8+10+1}$
	normal	5	60	50	$\text{precision}_n = \frac{60}{5+60+50}$
	spam	3	30	200	$\text{precision}_s = \frac{200}{3+30+200}$
		$\text{recall}_u = \frac{8}{8+5+3}$	$\text{recall}_n = \frac{60}{10+60+30}$	$\text{recall}_s = \frac{200}{1+50+200}$	

Microaveraging vs Macroaveraging

How to combine P/R values for different classes:

Class 1: Urgent

	true urgent	true not
system urgent	8	11
system not	8	340

$$\text{precision} = \frac{8}{8+11} = .42$$

Class 2: Normal

	true normal	true not
system normal	60	55
system not	40	212

$$\text{precision} = \frac{60}{60+55} = .52$$

Class 3: Spam

	true spam	true not
system spam	200	33
system not	51	83

$$\text{precision} = \frac{200}{200+33} = .86$$

Pooled

	true yes	true no
system yes	268	99
system no	99	635

$$\text{microaverage precision} = \frac{268}{268+99} = .73$$

$$\text{macroaverage precision} = \frac{.42+.52+.86}{3} = .60$$

Conclusion

Text Classification: Explored methods to categorise written text into predefined classes, such as sentiment analysis or spam detection.

Naïve Bayes Classifier: Introduced a simple and effective probabilistic model based on Bayes' Theorem, assuming independence between features.

Bag of Words: Discussed text representation using word frequencies while ignoring grammar and order, crucial for training classifiers.

Laplace Smoothing: Covered how smoothing handles unseen words in the training data to avoid zero probabilities.

Evaluation Metrics: Introduced precision, recall, F1 score, and confusion matrices as key tools for assessing classifier performance.